escola da nuvem.

aws certified Developer ASSOCIATE

# ormação Completa

# AWS Developer Associate



# Análise de Causa Raiz e Consulta de Logs na AWS

# Objetivo da Apresentação

### Capacitação

Aprender a rastrear falhas até sua origem usando ferramentas de análise de logs

### Interpretação

Compreender códigos HTTP e exceções lançadas pelos SDKs da AWS

### Otimização

Utilizar queries avançadas em ferramentas como o Amazon CloudWatch

### Cálculo de Capacidade

Compreensão das unidades de leitura e escrita (RCU e WCU) no DynamoDB






# Análise de Códigos HTTP

## Códigos 200

Indicam sucesso. A operação foi executada conforme esperado.

## Códigos 400

Apontam erros na requisição. Problema nos dados ou parâmetros enviados.

## Códigos 500

Refletem erros no servidor. Investigar infraestrutura ou processamento interno.



# Importância dos Códigos HTTP

## Identificação Rápida

Fundamental para determinar se a operação foi bem-sucedida ou falhou.

## Diferenciação de Erros

Crucial distinguir erros de cliente versus servidor para direcionar soluções.

## Estratégias Eficazes

Contribui para implementação de mecanismos de retry e fallback.

# Exemplo Prático: Análise de Códigos HTTP

```
import requests
response = requests.get('https://api.exemplo.com/dados')
if response.status_code == 200:
    print("Requisição bem-sucedida:", response.json())
elif response.status_code == 404:
    print("Recurso não encontrado.")
elif response.status_code >= 500:
    print("Erro no servidor. Tente novamente mais tarde.")
else:
    print("Código de status:", response.status_code)
```






# Aplicação Prática no Diagnóstico

### Monitoramento Integrado

Integrar análise de códigos HTTP com ferramentas como CloudWatch.

### Lógicas de Retry

Implementar retentativas baseadas na análise dos códigos de erro.

### Melhoria Contínua

Contribuir para performance e estabilidade dos sistemas.



# Benefícios da Análise de Códigos HTTP

### Eficiência

Melhora na identificação e correção rápida de falhas.

### Contribuição

Contribui para sistemas mais resilientes e estáveis.

### Certificação

Prática essencial para a certificação AWS Developer.

# Tratamento de Exceções no SDK



## Robustez

Manter aplicações estáveis durante interações com AWS



## Exceções Específicas

ResourceNotFound, AccessDenied, ThrottlingException



## Estratégias

Implementar retry ou fallback para evitar interrupções

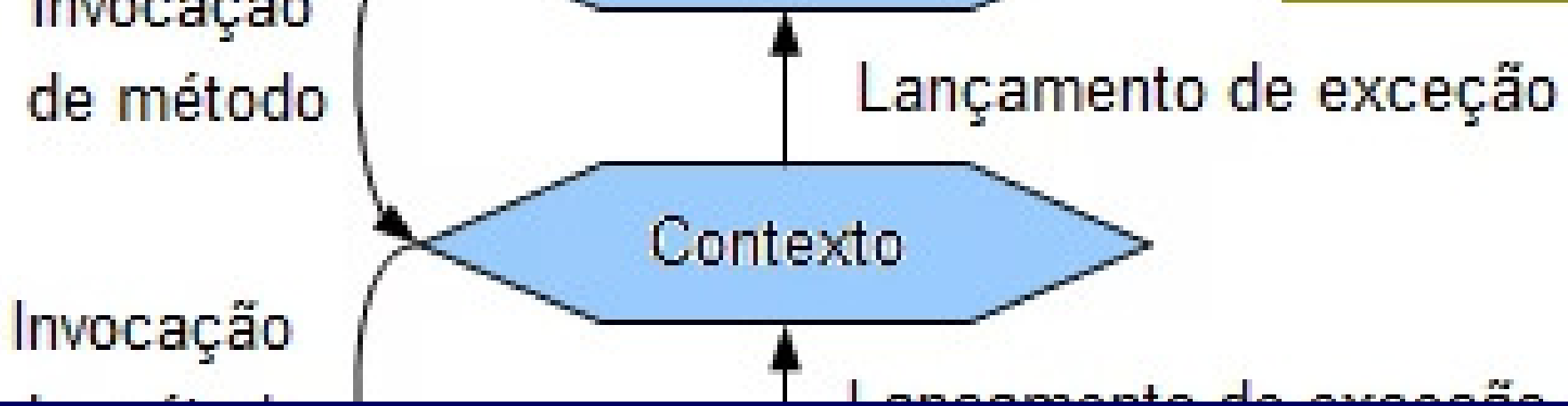
```
    int linhas = 0;
    BufferedReader br = new BufferedReader(
        new FileReader(arquivo));
    :
    return linhas;
}
```

# Exemplo: Tratamento de Exceções no Boto3

```
import boto3
from botocore.exceptions import ClientError

dynamodb = boto3.resource('dynamodb')
table = dynamodb.Table('MinhaTabela')

def inserir_item(item):
    try:
        table.put_item(Item=item)
        return "Inserido com sucesso"
    except ClientError as e:
        if e.response['Error']['Code'] == 'ProvisionedThroughputExceededException':
            print("Limite de throughput excedido. Tente novamente.")
            # Implementar lógica de retry aqui
        else:
            print("Erro inesperado:", e)
            raise
```




# Aspectos Essenciais do Tratamento de Exceções

**1** **Captura Específica**

Capture exceções específicas para implementar estratégias adequadas.

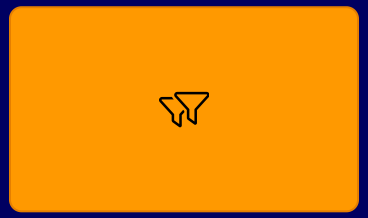
**2** **Recuperação Automática**

Permita que o sistema se recupere de falhas temporárias.

**3** **Conhecimento Detalhado**

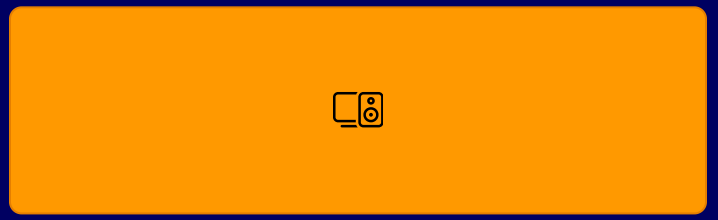
Entenda os tipos de exceção e a lógica apropriada para cada situação.

# Relembre: Estratégias de Tratamento de Exceções



## Identificação

Reconhecer o tipo específico de exceção



## Retry

Implementar tentativas com backoff exponencial



## Fallback

Utilizar alternativas quando a operação principal falha



# CloudWatch: Além do Básico

### Coleta

Agregação de logs de múltiplas fontes

### Alertas

Configuração de notificações baseadas em padrões

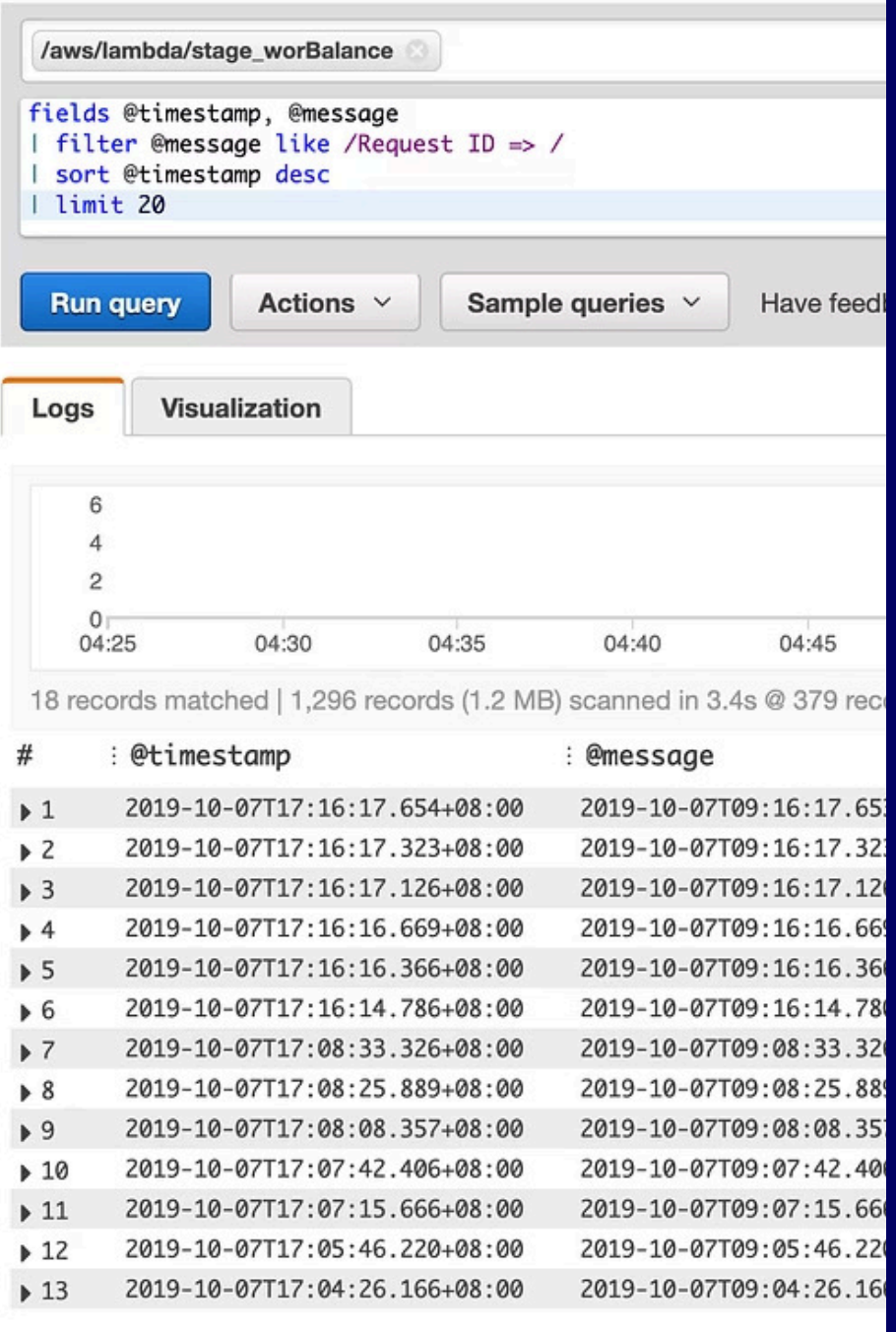


### Filtragem

Uso de queries para extrair informações relevantes

### Visualização

Criação de dashboards personalizados




# Uso de Queries no CloudWatch

## Filtragem Avançada

Filtrar logs por palavras-chave, datas e outros parâmetros específicos.

## Identificação de Padrões

Auxiliar na descoberta de tendências e comportamentos anômalos.

## Dashboards Customizados

Facilitar a criação de painéis visuais e alertas baseados nos resultados.

# Implementação de Queries via SDK

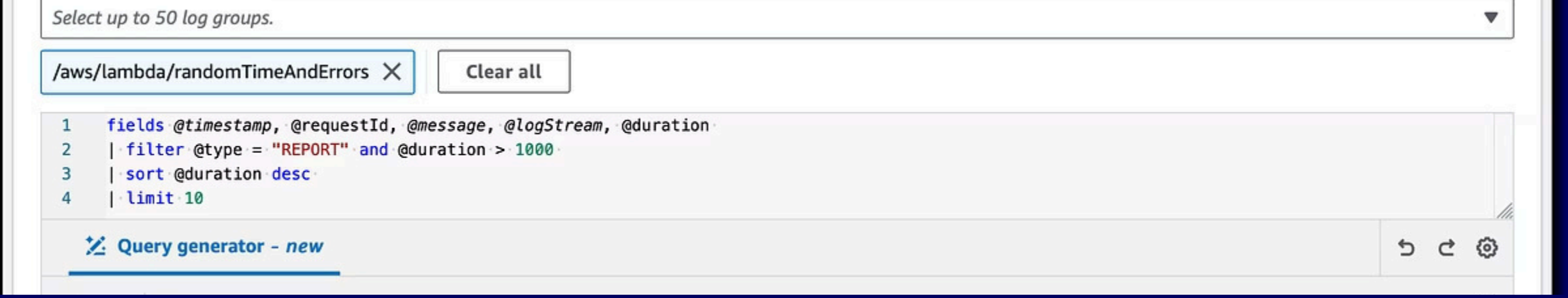
```
import boto3
import time

client = boto3.client('logs')
query = "fields @timestamp, @message | filter @message like /500/ | stats count() as errorCount by bin(1h)"

start_query_response = client.start_query(
    logGroupName='MinhaLogGroup',
    startTime=1622505600,  # Timestamp Unix
    endTime=1622592000,    # Timestamp Unix
    queryString=query,
)

query_id = start_query_response['queryId']
time.sleep(10)
response = client.get_query_results(queryId=query_id)
print("Resultados da query:", response['results'])
```

Esta query filtra mensagens contendo "500" (erro de servidor) e conta ocorrências por hora.




# Interpretação e Ajuste de Queries

### Ajuste Temporal

Defina parâmetros de tempo para focar em períodos relevantes.

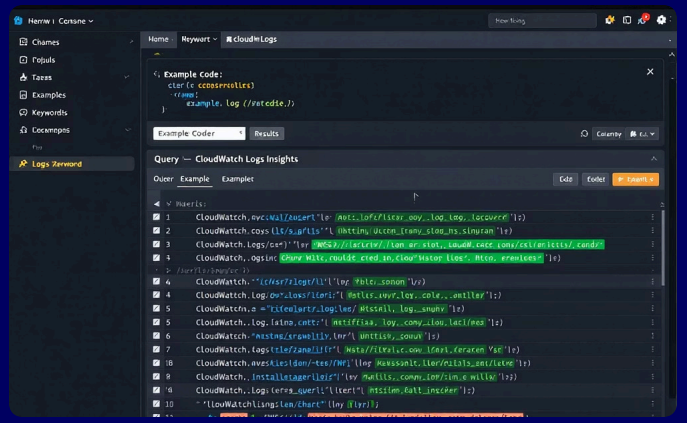
### Refinamento de Filtros

Ajuste filtros para capturar eventos específicos de interesse.

### Integração com Dashboards

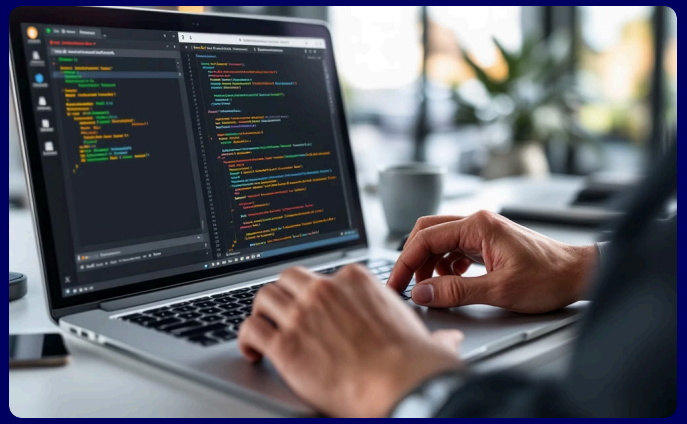
Incorpore resultados em painéis para monitoramento contínuo.

# CloudWatch Logs Insights









## Interface de Consulta

Editor poderoso com sintaxe específica para análise de logs.

## Visualização

Transforme resultados de consultas em gráficos e tabelas.

## Alertas

Configure notificações baseadas em padrões identificados nas consultas.

**Leia na documentação oficial:**

CloudWatch Registros da Amazon

**Análise de dados de registro com o CloudWatch Logs Insights – CloudW...**

Monitore e armazene seus arquivos de log a partir de AWS fontes. Em seguida, você pode recuperar os dados de registro associados do CloudWatch Logs.

# Exemplo: Usando o CloudWatch Logs Insights

Aqui temos um exemplo em Python que utiliza a biblioteca boto3 para listar os grupos de logs e executar uma query simples usando o CloudWatch Logs Insights

```python
import boto3
import time

# Cliente do CloudWatch Logs
logs_client = boto3.client('logs')

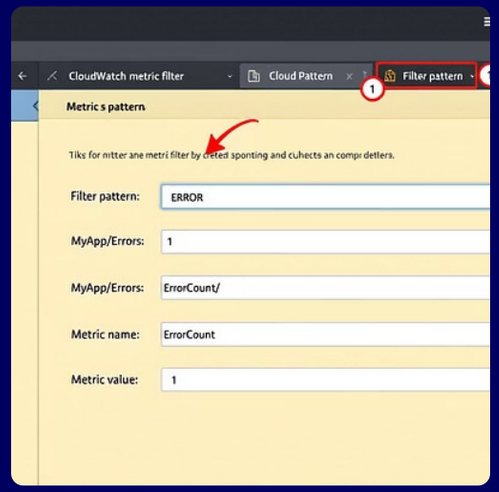
# Listar grupos de logs
groups = logs_client.describe_log_groups()
print("Grupos de logs:", groups['logGroups'])

# Iniciar uma query para filtrar logs que contenham a palavra "ERROR"
query_string = "fields @timestamp, @message | filter @message like /ERROR/"
start_query_response = logs_client.start_query(
    logGroupName='my-log-group',
    startTime=int(time.time()) - 3600,  # últimos 60 minutos
    endTime=int(time.time()),
    queryString=query_string,
)
query_id = start_query_response['queryId']

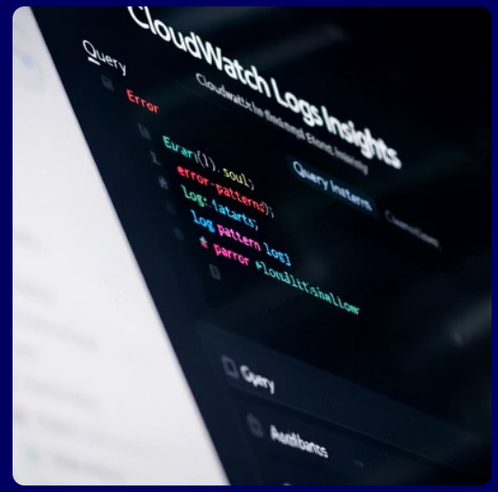
# Aguardar alguns segundos para a execução da query
time.sleep(5)

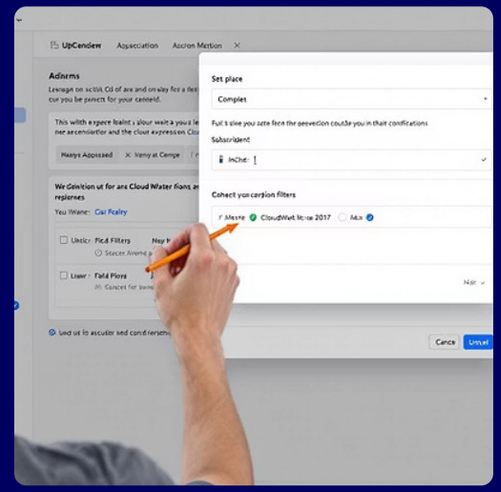
# Recuperar os resultados da query
results = logs_client.get_query_results(queryId=query_id)
print("Resultados da query:", results)
```

# CloudWatch Logs: Resumo





## Recursos Avançados

- **Filtros Métricos:** Extraem métricas de logs com base em padrões, como contagem de erros.
- **Dashboards Personalizados:** Visualize métricas dos logs em painéis em tempo real.
- **Logs Insights:** Ferramenta para análise avançada de logs com consultas personalizadas.
- **Filtros de Assinatura:** Enviam logs automaticamente para outros serviços como Lambda ou Kinesis para processamento em tempo real.

# Relembre: Amazon DynamoDB

O Amazon DynamoDB é um serviço de banco de dados NoSQL totalmente gerenciado que oferece performance previsível e escalabilidade. Sua estrutura é composta por elementos fundamentais que sustentam sua robustez.

aws

Amazon Web Services, Inc.

**Banco de dados de chave-valor NoSQL r...**

O Amazon DynamoDB é um banco de dados de chave-valor NoSQL, sem servidor e totalment...



# Componentes principais do Amazon DynamoDB

### Tabelas

Coleções de itens de dados organizados sem esquema rígido. Cada tabela requer apenas uma chave primária.

### Itens

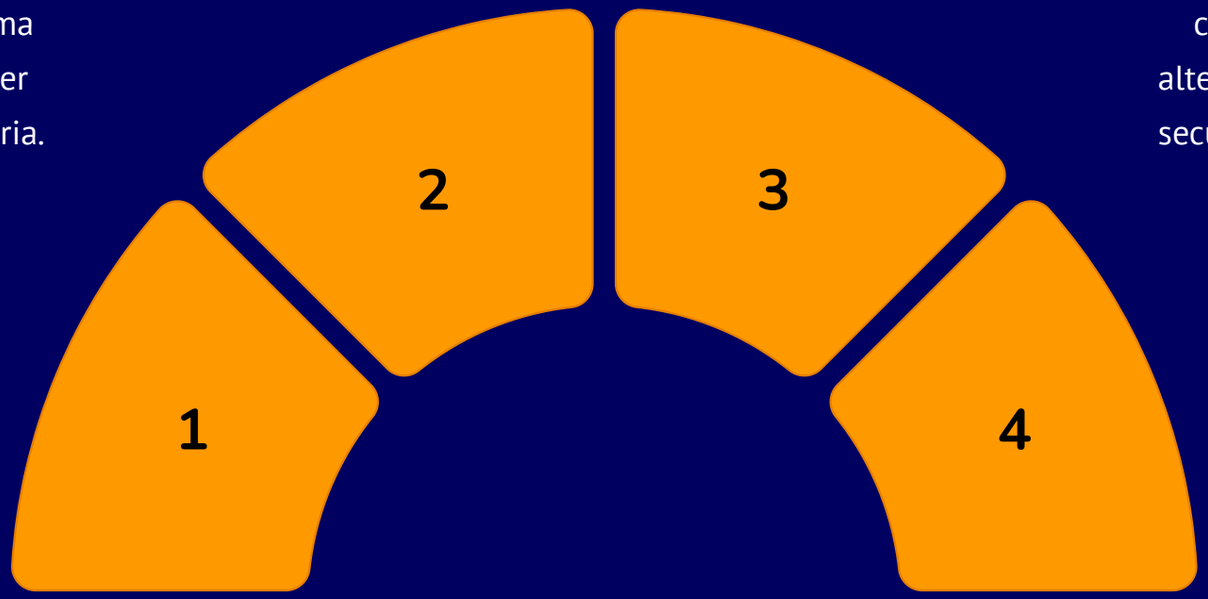
Registros individuais dentro de tabelas. Comparáveis às linhas em bancos relacionais, mas com estrutura flexível.

### Atributos

Elementos fundamentais de dados que compõem cada item. Podem ter tipos variados como string, número ou binário.

### Índices

Estruturas que otimizam consultas por caminhos alternativos. Incluem índices secundários locais e globais.




O DynamoDB utiliza unidades de capacidade de leitura (RCUs) e escrita (WCUs) para dimensionar o throughput necessário para sua aplicação.

# Cálculo de Capacidade no DynamoDB

É o processo de estimar quantas unidades de leitura (RCUs) e gravação (WCUs) sua tabela precisa para lidar com a carga esperada, considerando o tamanho dos itens e a frequência de acesso. Isso garante desempenho estável e evita erros por excesso de uso.



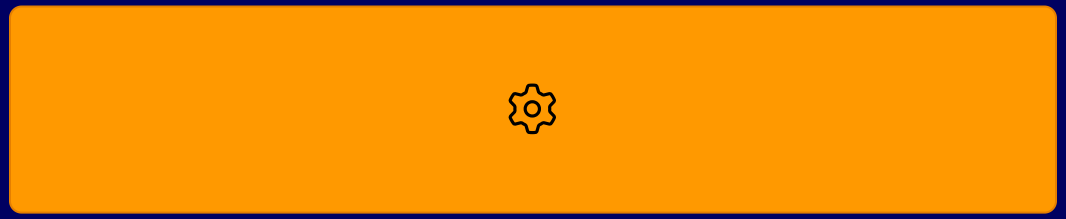
### Compreender Unidades de Capacidade

RCUs para leituras e WCUs para escritas



### Analisar Padrões de Acesso

Entender frequência e volume de operações



### Provisionar Adequadamente

Balancear custo e performance

# Unidade de Capacidade de Leitura (RCU) no DynamoDB

**RCU (Read Capacity Unit)**

- Cada RCU permite **1 leitura fortemente consistente por segundo** de item até **4 KB**.

## Tipos de leitura

| Fortemente consistente | Eventualmente consistente | Transacional |
| --- | --- | --- |
| • 4 KB = 1 RCU \| 8 KB = 2 RCUs | • 1 RCU = **2 leituras por segundo** (mais eficiente)<br>• 8 KB = 1 RCU | • Usa o **dobro das RCUs do Fortemente consistentes.**<br>• 4 KB = 2 RCU \| 8 KB = 4 RCUs |

# Unidade de Capacidade de Gravação (WCU) no DynamoDB

## WCU (Write Capacity Unit)

Cada WCU permite **1 gravação por segundo** de um item de até **1 KB**.

- Itens maiores usam mais WCUs (ex: 3 KB = 3 WCUs).
- Gravações **transacionais** usam **o dobro de WCUs** (ex: 3 KB = 6 WCUs).

## rWCU (Replicated WCU)

Usada em **tabelas globais**, pois cada gravação é replicada em **outras regiões** da AWS.

- Cada região replicada também consome WCUs.

# Cálculo de RCUs e WCUs

Neste slide, a veremos como calcular a capacidade provisionada no DynamoDB, considerando o tamanho dos itens e a carga de leitura e gravação esperada.

## Cenário:

- **Tamanho médio do item:** 7 KB
- **Carga esperada:** 100 leituras/s e 50 gravações/s

## Cálculos:

- **Leitura fortemente consistente:** 7 KB → 2 RCUs por leitura → **200 RCUs**
- **Leitura eventualmente consistente:** 7 KB → 1 RCU por leitura → **100 RCUs**
- **Gravação:** 7 KB → 7 WCUs por gravação → **350 WCUs**

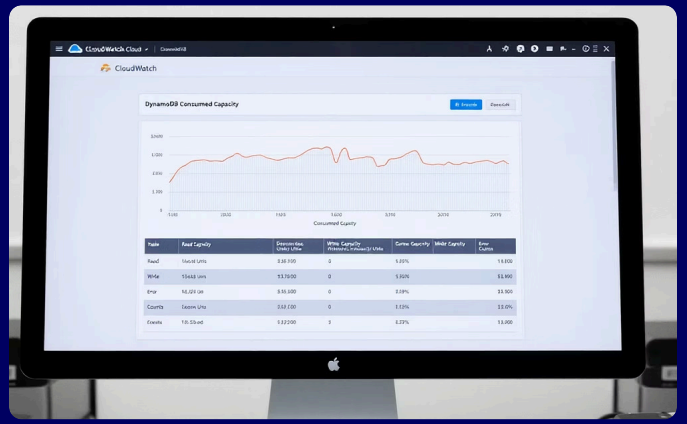
## Resultado:

- **Provisionamento necessário:**
  - **200 RCUs** (ou 100 RCUs, se eventual)
  - **350 WCUs**

| Tipo de Operação | Tamanho do Item | Unidades por Operação | Operações por Segundo | Total Necessário |
|---|---|---|---|---|
| **Leitura (forte)** | 7 KB | 2 RCUs | 100 | **200 RCUs** |
| **Leitura (eventual)** | 7 KB | 1 RCU | 100 | **100 RCUs** |
| **Gravação** | 7 KB | 7 WCUs | 50 | **350 WCUs** |

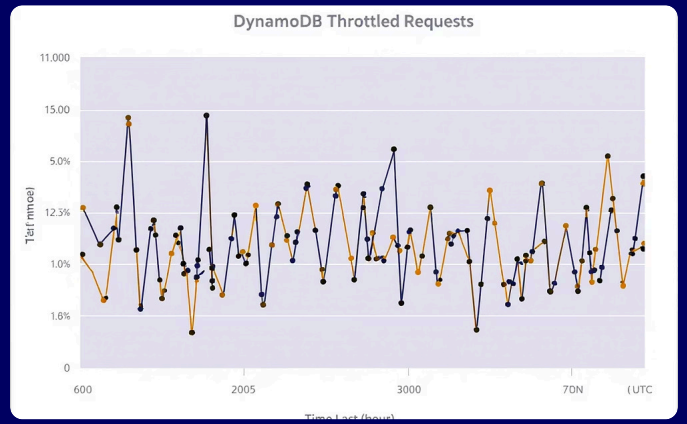
**Dica:** Use o modo sob demanda se a carga for variável, a AWS gerencia as unidades automaticamente.

# Monitorando Capacidade do DynamoDB



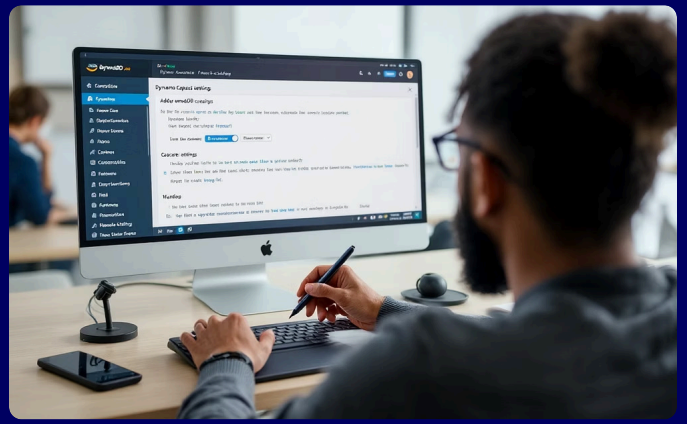

## Métricas de Consumo

Monitore ConsumedReadCapacityUnits e ConsumedWriteCapacityUnits.




## Operações Limitadas

Acompanhe ReadThrottleEvents e WriteThrottleEvents para identificar gargalos.



## Ajuste Dinâmico

Utilize auto scaling para ajustar capacidade conforme demanda.

# Sincronização de Data/Hora em Instâncias EC2

### Importância

Crucial para logs precisos, transações e segurança. Evita problemas de autenticação.

### NTP

Network Time Protocol mantém sincronização precisa com servidores de tempo confiáveis.

### Amazon Time Sync Service

Serviço da AWS disponível em 169.254.169.123, baseado em relógios atômicos.



# Sincronização de Tempo com AWS Time Sync (Amazon Linux 2)

```
# Instala o chrony
sudo yum install chrony -y

# Configura o serviço para usar o servidor de tempo interno da AWS
sudo cat << EOF > /etc/chrony.conf
server 169.254.169.123 prefer iburst
driftfile /var/lib/chrony/drift
makestep 1.0 3
logdir /var/log/chrony
EOF

# Reinicia o serviço de sincronização
sudo systemctl restart chronyd
```

```
# Sincroniza com servidores públicos de NTP
sudo ntpdate -u pool.ntp.org

# Verifica a data e hora atual do sistema
date
```

| Comando | Instalação/Sincronização? | Mantém sincronizado? | Recomendado? |
|---|---|---|---|
| **chrony** | Instala e mantém | ✅ Sim | ✅ Sim (moderno e confiável) |
| **ntpdate** | Apenas sincroniza pontualmente | ❌ Não | ⚠️ Só para ajuste manual rápido |

# Verificando a Sincronização de Tempo

## Verificação do Status

```
# Verificar status
chronyc tracking

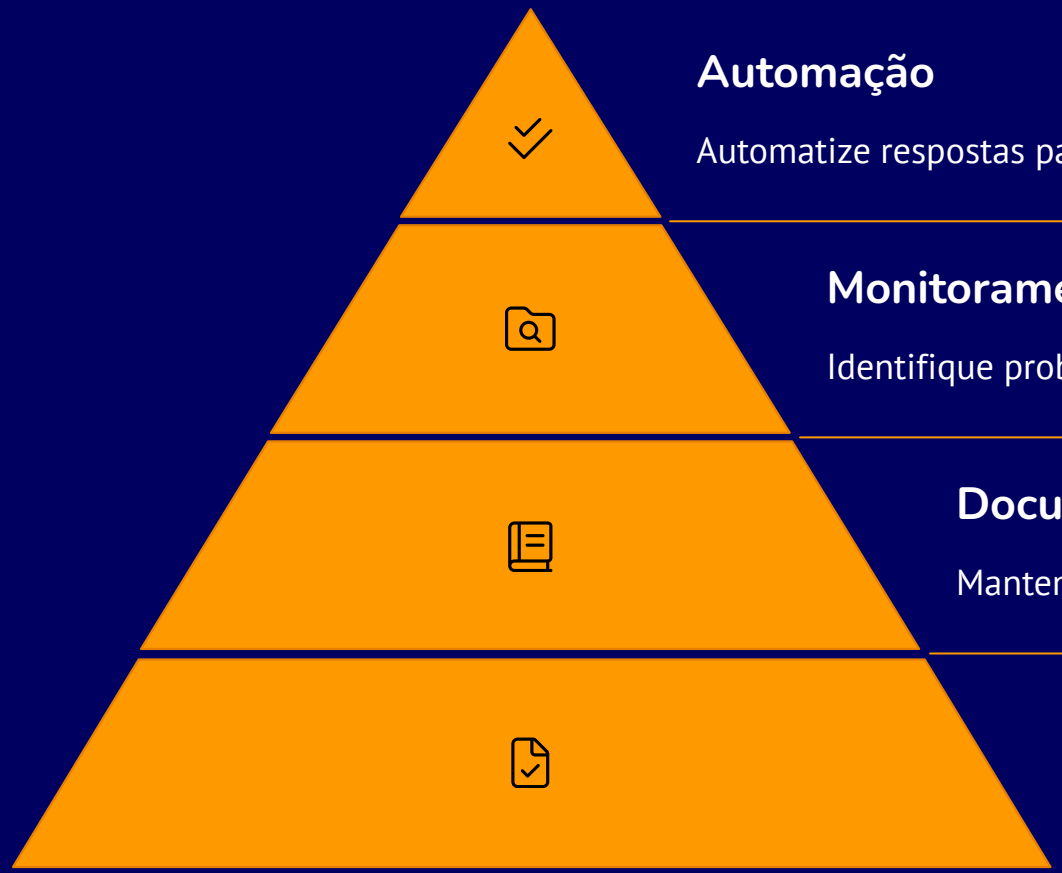
# Listar fontes
chronyc sources -v

# Verificar sincronização
timedatectl status
```

## Benefícios

- Logs com timestamps precisos
- Transações consistentes
- Autenticação confiável
- Coordenação entre serviços

# Melhores Práticas



## Automação

Automatize respostas para problemas comuns

## Monitoramento Proativo

Identifique problemas antes que afetem usuários

## Documentação

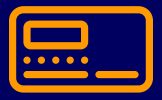
Mantenha registros de incidentes e soluções

## Testes Regulares

Verifique sistemas de recuperação periodicamente

# Por que isso é Importante

# (Visão Prática e da Certificação)



### Performance

Diagnóstico rápido mantém sistemas funcionando com eficiência.



### Confiabilidade

Tratamento adequado de erros aumenta a robustez das aplicações.



### Custo

Dimensionamento correto de recursos evita gastos desnecessários.



### Experiência

Sistemas estáveis proporcionam melhor experiência aos usuários.

# Visão para Certificação

## Consulta de Logs

Essencial para diagnosticar comportamentos inesperados e coletar evidências durante incidentes. Ferramentas envolvidas: Amazon CloudWatch Logs, AWS X-Ray, entre outras. Compreender como acessar, estruturar e analisar logs é fundamental para a observabilidade de aplicações.

## Root Cause Analysis (RCA)

Capacidade de identificar a origem real de falhas. Envolve correlação de logs, tracing, métricas e eventos. Requer entendimento sobre a interação entre serviços e componentes da aplicação em ambientes distribuídos.




## Cálculo de Capacidade no DynamoDB

Fundamental para otimizar desempenho e custo. Exige compreensão das unidades de leitura e escrita (RCU e WCU), padrão de acesso aos dados e tamanho dos itens. Inclui estratégias para uso eficiente em modo sob demanda ou provisionado.